

Sous-etape 3-i, et la sonde qui ne couvrait rien

Force Tracker — 12/09/2026, ft-v1196. Huitieme document de la serie. Michel : « *continue selon le decoupage, une sous-etape a la fois, en gardant exactement les memes regles* ». **La sous-etape est petite ; ce qui s'est passe AVANT de la coder l'est moins.**

LE FAIT PRINCIPAL DE CETTE VERSION

Mon propre document annonçait « *instantane : couvert* » pour 3-i. Mesure en ouvrant la sonde : a moitié faux.

Un seul des deux sites etait reellement couvert, et la sonde qui portait le nom de la regle **recopiait cette regle chez elle** au lieu d'appeler la production.

Corrige AVANT toute ligne de code, comme la regle de decoupage l'exige : « *l'instantane couvre deja ses sites, OU on ecrit la sonde AVANT* ».

1. Ce que 3-i devait faire

La suite naturelle de 1b-i : **le meme couple de fonctions** (`rejouerRepas · quickAddFood`), **l'autre moitié de la ligne**. Le bloc `{q, u, per100, ...}` a son proprietaire depuis ft-v1195 ; le **TEST** qui decide si la quantite passe restait ecrit deux fois.

```
// AVANT – le meme test, ecrit deux fois

// jouerRepas
const qOk=(+e.q>0 && (!e.u||e.u==='g'||e.u==='portion'));

// quickAddFood
const _qOk=(+it.q>0 && (!it.u||it.u==='g'||it.u==='portion'));

// APRES – un proprietaire, et il rend un BOOLEEN sans rien toucher

function _qRepreable(src){
  const s = src || {};
  return (+s.q > 0 && (!s.u || s.u === 'g' || s.u === 'portion'));
}

const qOk = _qRepreable(e); // jouerRepas
const _qOk = _qRepreable(it); // quickAddFood
```

2. Ce que la regle dit, et ce qu'elle refuse

cas	verdict	pourquoi
{q:120, u:'g'}	oui	des grammes, on sait redimensionner
{q:2, u:'portion'}	oui	c'est la moitié « avec portions » (ft-v1183/1186)
{q:80, u:null}	oui	_srcRepriseQ retombe alors sur 'g'
{q:0, u:'g'}	non	une quantite nulle n'est pas une quantite
{q:-5, u:'g'}	non	idem, dans l'autre sens
{q:150, u:'ml'}	non	sans densite, un volume ne dit pas ce que PESE l'aliment — et on n'invente pas une densite (R29)

Elle rend un boolean et ne touche a rien : c'est _srcRepriseQ qui decide quoi en faire. Une sous-etape reversible est une sous-etape qui ne fait qu'une chose.

3. La sonde qui ne couvrait rien

```
// LA SONDE QUI NE COUVRAIT RIEN — mesuree AVANT de coder

const regleP = c => (+c.q>0 && (!c.u||c.u==='g'||c.u==='portion')); // <- DANS la sonde
out['3_regle_avec_portions'] = J(CAS.map(c => [c.q, c.u, regleP(c)]));

// Elle n'appelle AUCUN code de production : elle ne peut rien detecter d'une extraction.
// C'est une table de verite, pas une couverture.

// Et la seule sonde qui conduisait une vraie porte n'en conduisait qu'UNE sur deux :
out['3_via_quickAddFood'] = ... // quickAddFood
// rejouerRepas : rien.
```

POURQUOI CE N'EST PAS UN DETAIL

Le critere de reussite de chaque sous-etape est **binaire** : *l'instantane doit etre identique octet pour octet avant et apres*. **Un instantane qui ne conduit pas la production ne peut pas remplir ce role** — il resterait identique quoi qu'on fasse au code.

Une sonde qui recopie la regle mesure ce qu'on CROYAIT écrire, pas ce qui est execute.

Corrige : la sonde 3_via_rejouerRepas comble la moitié manquante, et l'instantane passe de **11** a **12 cles**. Le BEFORE a ete capture **avec la sonde etendue** — l'etendre apres l'extraction aurait donne un avant / apres **incomparable**.

ET LE SIGNE ETAIT DANS LE COMMENTAIRE LUI-MEME

Il annonçait « *les six sites conduits par leur VRAIE porte* » pour une boucle qui n'en conduit **qu'une**.

=> **C'est le miroir du commentaire de ft-v1190**, qui annonçait une portee plus **ETROITE** que le code. *Dans les deux sens, un commentaire qui decrit mal sa portee dispense le lecteur suivant d'aller verifier.*

Ecrit dans BUGS.md §58 (*verifier la fonction n'est pas verifier l'appel*), dont c'est la version **cote SONDE** : le reflexe est **d'ouvrir la sonde** et d'y chercher le nom de la fonction de production. S'il n'y est pas, elle ne couvre rien.

4. Ce que 3-i n'a PAS fait, et c'est la moitié qui compte

LES 5 SITES « GRAMMES SEULS » SONT INTACTS

Ils refusent les portions **expres** : ils alimentent un champ **en grammes**.

Les deux regles se ressemblent a un | | pres et ne disent pas la meme chose. Les fondre serait un **changement de comportement**, pas une extraction — c'est la sous-etape **3-ii / 3-iii / 3-iv**, et l'une des **4 decisions produit** qui attendent Michel.

Un temoin de perimetre exige que ce compte reste a 5. Le jour ou il tombe a 1, c'est que les sous-etapes suivantes ont ete faites — et ce temoin devra alors se **DEPLACER**, pas disparaître.

5. Le temoin de perimetre de 1b-i s'est DEPLACE, pas supprime

en 1b-i (ft-v1195)	en 3-i (ft-v1196)
les deux portes calculent ENCORE q0k chacune dans son corps (le garde-fou qui empechait 1b-i de deborder sur l'etape 3)	la regle existe a UN SEUL endroit, et les deux portes L'APPELLENT (3-i est precisement la sous-etape qui retire le premier)

=> **La difference entre un temoin qu'on retire et un temoin qui se deplace se mesure** : les deux mutations qui le faisaient rougir en 1b-i (une porte qui delegue · la regle retiree d'une porte) le font **toujours** rougir, par l'autre bout.

6. Les mesures

preuve	resultat
Instantane (12 cles), avant / apres	identique octet pour octet , sha256 d5b0572cafcc4477 <i>(la sha annoncee dans le message de commit, 64099b39025027ec, etait fausse : elle venait d'une version intermediaire de la sonde — verifie en jouant la sonde actuelle sur l'app.js d'AVANT 3-i, meme sha des deux cotes, diff vide)</i>
Controle negatif	10 mutations, TOUTES mordent
Source : _qReprenable appele	3 occurrences (1 declaration + 2 appelants), regle ecrite 1 fois
Source : perimetre	regle « grammes seuls » toujours ecrite 5 fois

DEUX PIEGES D'OUTILLAGE, REPAYES

(1) La passe de REFERENCE a attrape une regex trop stricte — avant la moindre mutation. Mon temoin de perimetre comptait les 5 sites avec `\|\|` sans espaces ; `or_provFood @1232` ecrit `( !_afSrc.u || _afSrc.u==='g' )` avec des espaces, les quatre autres sans. Il comptait **4 au lieu de 5** et serait parti rouge en passe complete.
=> **Un temoin de source se verifie d'abord contre le code SAIN : s'il rougit la, il ne mesure pas ce qu'il croit.**

(2) Une mutation a encore TUE la sonde au lieu de la faire rougir (§61, 3^e fois recensee) : retirer le garde `src || {}` fait lever `_qReprenable(null)`, l'evalaute entier est rejete et le bloc disparaît sans rougir. Temoin mis sous try/catch avec un message nomme — la mutation fait désormais **1 rouge, exactement lui**.

7. Ce qui reste

sujet	etat
Les 8 sous-etapes restantes	ecrites, ordonnees et dependancees dans docs/SOUS-ETAPES-1B-3.md. // 1b-ii et 1b-iv portent un PREREQUIS de sonde — et le cas de 3-i montre qu'il faut ouvrir la sonde pour en juger, pas lire l'etiquette
Les 4 harmonisations	decisions produit , elles attendent Michel. Critere : <i>est-ce que ca modifie ce qui est ECRIT dans S. foodLog / S. savedFoods ?</i>
Le hub (etape 4) et la douane (etape 5)	apres 1b et 3
S. savedFoods · l'ecart 48,3 / 48	ouverts , non corriges
L'historique, les migrations	non touches

Document genere depuis le code et la sonde servis — **tous les decomptes cites sont recomptes a chaque generation** (3 appels, 1 ecriture de la regle, 5 sites grammes, 2 portes, 12 cles de sonde). **Huit gardes** refusent de produire si un fait tombe — dont un qui verifie que le constat du §3 est **encore vrai**, et un qui verifie que le perimetre n'a **pas** deborde. // **Et on compte les CLES DISTINCTES de l'instantane, pas les lignes** (13 affectations pour 12 cles : une cle est assignee dans un try et dans son catch) — *compter les lignes aurait ete exactement l'erreur que ce document denonce*. Source : `tools/gen_3i_pdf.py`.